Documentación Técnica: Diseño, Maquetación Headless e Integración Frontend del Sistema de Identidad Keycloak para OpenIoTData

Proyecto: OpenIoTData — Portal Abierto de Datos

Módulo: Autenticación y Autorización (OIDC / OAuth2)

Fecha: 20 de Octubre de 2026

1. Justificación y Contexto Técnico

En el diseño original del sistema OpenIoTData, la arquitectura de servicios completa contempla el despliegue mediante orquestación de contenedores Docker (incluyendo PostgreSQL/PostGIS, MinIO, Celery Worker, ClamAV, Nginx y Keycloak).

Sin embargo, para las actividades de maquetación de vistas en el *frontend* (React) e integración de endpoints de gestión de usuarios en el *backend* (Python), el entorno de desarrollo local presenta una restricción estricta de memoria RAM (**4 GB**).

Ejecutar simultáneamente Docker Desktop, el motor WSL 2 y la suite completa de contenedores sobrepasa la capacidad de memoria del equipo, generando congelamiento del sistema e imposibilitando el flujo de trabajo.

Por ello, se definió una **estrategia de desarrollo nativo ligero (sin Docker)**:

1.1 **Script de desarrollo (dev.mjs):** Levanta el Backend API en Python (http://localhost:8000) y el Frontend React (http://localhost:3000), usando **SQLite** localmente para migraciones rápidas con Alembic (alembic upgrade head).

```
1. #!/usr/bin/env node
2. // Lanzador del modo host de OpenIoTData: prepara el entorno y arranca
   la API y el
3. // frontend sin Docker. Compatible con Windows, macOS y Linux
   (requiere Node.js 22).
4. //
5. // Uso: node scripts/dev.mjs [--api-only|--web-only] [--no-seed] [--
   reset|--env|--help]
6.
7. import { spawn, spawnSync } from "node:child_process";
8. import { existsSync, mkdirSync, readFileSync, rmSync } from "node:fs";
9. import { dirname, join, resolve } from "node:path";
10.import { fileURLToPath } from "node:url";
11.
12.const rootDir = resolve(dirname(fileURLToPath(import.meta.url)),
   "..");
```

```
13. const backendDir = join(rootDir, "backend");
14. const frontendDir = join(rootDir, "frontend");
15. const dataDir = join(backendDir, "data");
16. const envPath = join(rootDir, "scripts", "dev-local.env");
17. const isWindows = process.platform === "win32";
18.
19. const flags = new Set(process.argv.slice(2));
20. const wantsHelp = flags.has("-h") || flags.has("--help");
21. const wantsEnv = flags.has("--env");
22. const wantsReset = flags.has("--reset");
23. const apiOnly = flags.has("--api-only");
24. const webOnly = flags.has("--web-only");
25. const noSeed = flags.has("--no-seed");
26.
27. function log(message) {
28.   console.log(`[dev] ${message}`);
29. }
30.
31. function fail(message) {
32.   console.error(`[dev] Error: ${message}`);
33.   process.exit(1);
34. }
35.
36. function printUsage() {
37.   console.log(`Modo host de OpenIoTData (backend + frontend, sin
    Docker)
38.
39. Uso: node scripts/dev.mjs [opciones]
40.
41.   (sin opciones)  Prepara el entorno, carga los datos demo y arranca
    API y frontend
42.   --api-only      Arranca sólo la API en http://localhost:8000
43.   --web-only      Arranca sólo el frontend en http://localhost:3000
44.   --no-seed       No carga el catálogo ilustrativo
45.   --reset         Elimina backend/data (base local y objetos) y
    termina
46.   --env           Muestra la configuración resuelta y termina
47.   -h, --help      Muestra esta ayuda
48.
49. Este modo omite ClamAV, Keycloak/OIDC, PostGIS y el worker Celery. No
    sustituye a
50. 'make validate-beta' ni debe usarse en entornos compartidos.`);
51. }
52.
53. function parseEnvFile(path) {
```

```
  const result = {};
  for (const rawLine of readFileSync(path, "utf8").split(/\r?\n/)) {
    const line = rawLine.trim();
    if (!line || line.startsWith("#")) continue;
    const separator = line.indexOf("=");
    if (separator === -1) continue;
    const key = line.slice(0, separator).trim();
    let value = line.slice(separator + 1).trim();
    if (
      (value.startsWith('"') && value.endsWith('"')) ||
      (value.startsWith("'") && value.endsWith("'"))
    ) {
      value = value.slice(1, -1);
    }
    result[key] = value;
  }
  return result;
}

function run(command, commandArgs, cwd, useShell = false) {
  const result = spawnSync(command, commandArgs, {
    cwd,
    env: hostEnv,
    shell: useShell,
    stdio: "inherit",
  });
  if (result.error) fail(`no se pudo ejecutar ${command}:
  ${result.error.message}`);
  if (result.status !== 0) fail(`${command} terminó con código
  ${result.status}`);
}

function pipeOutput(stream, label) {
  let buffer = "";
  stream.setEncoding("utf8");
  stream.on("data", (chunk) => {
    buffer += chunk;
    const lines = buffer.split(/\r?\n/);
    buffer = lines.pop() ?? "";
    for (const line of lines) {
      if (line.trim()) console.log(`${label} ${line}`);
    }
  });
  stream.on("end", () => {
    if (buffer.trim()) console.log(`${label} ${buffer}`);
```

```
97.   });
98. }
99.
100.     const children = [];
101.     let shuttingDown = false;
102.
103.     function shutdown(code) {
104.       if (shuttingDown) return;
105.       shuttingDown = true;
106.       log("deteniendo servicios...");
107.       for (const child of children) {
108.         if (child.exitCode !== null || child.killed) continue;
109.         if (isWindows) {
110.           spawnSync("taskkill", ["/pid", String(child.pid), "/T",
    "/F"], { stdio: "ignore" });
111.         } else {
112.           try {
113.             process.kill(-child.pid, "SIGTERM");
114.           } catch {
115.             try {
116.               child.kill("SIGTERM");
117.             } catch {
118.               /* proceso ya finalizado */
119.             }
120.           }
121.         }
122.       }
123.       setTimeout(() => process.exit(code), 800);
124.     }
125.
126.     function launch(label, command, commandArgs, cwd, useShell =
    false) {
127.       const child = spawn(command, commandArgs, {
128.         cwd,
129.         env: hostEnv,
130.         shell: useShell,
131.         detached: !isWindows,
132.         stdio: ["ignore", "pipe", "pipe"],
133.       });
134.       children.push(child);
135.       if (child.stdout) pipeOutput(child.stdout, label);
136.       if (child.stderr) pipeOutput(child.stderr, label);
137.       child.on("error", (error) => fail(`no se pudo iniciar
    ${label}: ${error.message}`));
138.       child.on("exit", (code) => {
```

```
139.         if (shuttingDown) return;
140.         log(`${label} finalizó con código ${code ?? "?"}; deteniendo
   el resto.`);
141.         shutdown(code ?? 1);
142.       });
143.       return child;
144.     }
145.
146.     function venvPythonCandidates() {
147.       return [join(backendDir, ".venv"), join(backendDir,
   ".venv312")].map((dir) =>
148.         isWindows ? join(dir, "Scripts", "python.exe") : join(dir,
   "bin", "python"),
149.       );
150.     }
151.
152.     function existingVenvPython() {
153.       return venvPythonCandidates().find((candidate) =>
   existsSync(candidate)) ?? null;
154.     }
155.
156.     function detectPython() {
157.       const candidates = isWindows
158.         ? [["py", ["-3.12"]], ["python", []], ["python3", []]]
159.         : [["python3.12", []], ["python3", []], ["python", []]];
160.       for (const [command, baseArgs] of candidates) {
161.         const probe = spawnSync(command, [...baseArgs, "--version"],
   { encoding: "utf8" });
162.         if (probe.error || probe.status !== 0) continue;
163.         const match = `${probe.stdout ?? ""}${probe.stderr ??
   ""}`.match(/Python (\d+)\.(\d+)/);
164.         if (!match) continue;
165.         const major = Number(match[1]);
166.         const minor = Number(match[2]);
167.         if (major === 3 && minor >= 12) return { command, baseArgs,
   version: `${major}.${minor}` };
168.       }
169.       return null;
170.     }
171.
172.     function hasBackendDeps(python) {
173.       const probe = spawnSync(
174.         python,
175.         ["-c", "import fastapi, uvicorn, duckdb, pyarrow,
   sqlalchemy, aiosqlite"],
```

```
176.         { cwd: backendDir, env: hostEnv, stdio: "ignore" },
177.       );
178.       return probe.status === 0;
179.     }
180.
181.     function ensureBackend() {
182.       mkdirSync(dataDir, { recursive: true });
183.       let python = existingVenvPython();
184.       if (!python) {
185.         const found = detectPython();
186.         if (!found) fail("no se encontró Python 3.12 o superior.
    Instálelo y vuelva a intentar.");
187.         log(`creando backend/.venv con ${found.command} (Python
    ${found.version})...`);
188.         run(found.command, [...found.baseArgs, "-m", "venv",
    join(backendDir, ".venv")], rootDir);
189.         python = existingVenvPython();
190.         if (!python) fail("no se pudo crear backend/.venv");
191.         log("instalando dependencias del backend...");
192.         run(python, ["-m", "pip", "install", "--upgrade", "pip"],
    backendDir);
193.         run(python, ["-m", "pip", "install", "-r",
    "requirements.lock"], backendDir);
194.       } else if (!hasBackendDeps(python)) {
195.         log("instalando dependencias del backend...");
196.         run(python, ["-m", "pip", "install", "-r",
    "requirements.lock"], backendDir);
197.       }
198.       return python;
199.     }
200.
201.     function ensureFrontend() {
202.       const sentinel = join(frontendDir, "node_modules", "next",
    "package.json");
203.       if (existsSync(sentinel)) return;
204.       log("instalando dependencias del frontend (npm ci)...");
205.       run("npm", ["ci"], frontendDir, isWindows);
206.     }
207.
208.     function showEnvironment() {
209.       const keys = [
210.         "APP_ENV",
211.         "DATABASE_URL",
212.         "STORAGE_BACKEND",
213.         "LOCAL_STORAGE_PATH",
```

```
214.         "CELERY_TASK_ALWAYS_EAGER",
215.         "DEV_AUTH_BYPASS",
216.         "CLAMAV_REQUIRED",
217.         "NEXT_PUBLIC_API_URL",
218.         "NEXT_PUBLIC_DEV_AUTH_BYPASS",
219.       ];
220.       console.log("Configuración del modo host (scripts/dev-
   local.env):");
221.       for (const key of keys) {
222.         console.log(`  ${key.padEnd(28)} ${hostEnv[key] ?? ""}`);
223.       }
224.     }
225.
226.     if (wantsHelp) {
227.       printUsage();
228.       process.exit(0);
229.     }
230.
231.     if (apiOnly && webOnly) fail("--api-only y --web-only son
   mutuamente excluyentes.");
232.     if (!existsSync(envPath)) fail(`no se encontró ${envPath}`);
233.
234.     const hostEnv = { ...process.env, ...parseEnvFile(envPath) };
235.     if (hostEnv.APP_ENV === "production") {
236.       fail("el modo host no puede ejecutarse con
   APP_ENV=production.");
237.     }
238.
239.     log("modo host: sin ClamAV, sin OIDC/Keycloak, sin PostGIS y sin
   worker Celery.");
240.
241.     if (wantsReset) {
242.       if (existsSync(dataDir)) {
243.         rmSync(dataDir, { recursive: true, force: true });
244.         log("datos locales eliminados (backend/data).");
245.       } else {
246.         log("no hay datos locales que eliminar.");
247.       }
248.       process.exit(0);
249.     }
250.
251.     if (wantsEnv) {
252.       showEnvironment();
253.       process.exit(0);
254.     }
```

```
255.
256.     process.on("SIGINT", () => shutdown(0));
257.     process.on("SIGTERM", () => shutdown(0));
258.
259.     let python = null;
260.     if (!webOnly) python = ensureBackend();
261.     if (!apiOnly) ensureFrontend();
262.
263.     if (!webOnly && !noSeed) {
264.       log("cargando el catálogo ilustrativo (operación
   idempotente)...");
265.       run(python, ["-m", "app.seed"], backendDir);
266.     }
267.
268.     if (!webOnly) {
269.       launch(
270.         "[api]",
271.         python,
272.         ["-m", "uvicorn", "app.main:app", "--reload", "--host",
   "127.0.0.1", "--port", "8000"],
273.         backendDir,
274.       );
275.     }
276.     if (!apiOnly) {
277.       launch("[web]", "npm", ["run", "dev"], frontendDir,
   isWindows);
278.     }
279.
280.     log("API: http://localhost:8000   UI: http://localhost:3000");
281.     log("Pulse Ctrl+C para detener.");
282.
```

Justificación de los Cambios en dev-local.env

1.2. Desactivación del Modo de Autenticación Simulada (DEV_AUTH_BYPASS=false y NEXT_PUBLIC_DEV_AUTH_BYPASS=false)

- **Motivo:** En las etapas iniciales de desarrollo, se utilizaba un mecanismo de *bypass* para simular un usuario local predeterminado y omitir la pasarela de seguridad.
- **Justificación del cambio:** Para transicionar hacia una arquitectura de autenticación real basada en estándares de producción (OIDC), se cambió este parámetro a false tanto en el backend como en el frontend. Esto obliga a

la aplicación a delegar la gestión de identidades, tokens JWT y sesiones exclusivamente al servidor de Keycloak, garantizando un flujo de autenticación verídico y seguro.

1.3. Integración y Configuración del Proveedor OIDC (Keycloak)

- **Motivo:** El sistema requería conectarse a una instancia local de Keycloak (v26.6.2) gestionada en el entorno de desarrollo para resolver la autenticación y el registro de usuarios autónomos.
- **Justificación de los parámetros:**
    - NEXT_PUBLIC_OIDC_URL=http://openiotdata.localhost:8080/auth: Se configuró explícitamente con el subdominio local (openiotdata.localhost) y el contexto /auth. Esto asegura que tanto el frontend como la librería de integración (keycloak-js) construyan correctamente las rutas de redirección hacia los endpoints de autorización, inicio de sesión y registro, evitando errores de resolución de rutas (404 Not Found).
    - NEXT_PUBLIC_OIDC_REALM=openiotdata: Define el espacio de nombres (*realm*) dedicado dentro de Keycloak que contiene la estructura aislada de usuarios, roles y políticas de seguridad importadas para el proyecto.
    - NEXT_PUBLIC_OIDC_CLIENT_ID=openiotdata-web: Identifica de manera unívoca al cliente frontend (aplicación de página única pública bajo flujo PKCE) autorizado para interactuar con el realm de Keycloak.

2. **Keycloak Nativo:** Se desacopló del contenedor y se ejecuta de manera nativa utilizando el Runtime de Java (JDK 21) sobre la base de datos embebida H2. Esto reduce drásticamente el consumo de recursos a menos de 500 MB de RAM sin alterar el esquema OIDC definido para producción.

2. Pautas y Arquitectura del Servicio Nginx

En entornos de integración/producción (make validate-beta), **Nginx** actúa como Proxy Inverso y punto de entrada unificado:

- **Unificación de dominios y puertos:** Evita problemas de CORS (Cross-Origin Resource Sharing) enrutando:
    - / → Frontend React (:3000)
    - /api/ → Backend Python (:8000)
    - /auth/ → Keycloak (:8081 / :8080)
- **Terminación TLS/SSL:** Centraliza los certificados HTTPS.
- **Seguridad y límites de carga:** Protege contra solicitudes maliciosas y gestiona los límites de tamaño para carga de *datasets* pesados antes de derivarlos a MinIO/Celery.

3. Especificación del Realm de Autenticación (openiotdata-realm.json)

Se mantuvo y respetó íntegramente la especificación OIDC definida para la plataforma:

- **Realm Name:** openiotdata
- **Ruta relativa HTTP:** /auth
- **Roles de Aplicación:**
    - publisher: Permiso para crear y publicar *datasets* propios.
    - moderator: Permiso para revisar reportes y gestionar recursos en cuarentena.
- **Clientes configurados:**
    1. openiotdata-api: Cliente de tipo *bearerOnly* para la API en Python.
    2. openiotdata-web: Cliente de tipo *publicClient* para la SPA en React con soporte de *Standard Flow* y mapeo de audiencias (openiotdata-api-audience).
- **Usuarios de Prueba Preconfigurados:**
    - demo.publisher (Password: publisher-local-only) → Rol: publisher
    - demo.moderator (Password: moderator-local-only) → Roles: publisher, moderator

4. Procedimiento de Instalación y Despliegue Nativo

Para habilitar Keycloak sin Docker en el equipo de desarrollo, se ejecutaron los siguientes pasos en orden secuencial:

Paso 1: Configuración del Entorno Java (JDK)

1. Instalación **JDK 21 (LTS)**.
2. Definición de variables de entorno del sistema:
    - JAVA_HOME apuntando al directorio raíz del JDK ("C:\Program Files\Java\jdk-21.0.12.1")
    - Adición del directorio %JAVA_HOME%\bin a la variable de entorno PATH.
3. Verificación de la instalación:

Bash

java -version

Paso 2: Despliegue de Keycloak Nativo (v26.6.2)

1. Descarga y extracción del paquete comprimido de Keycloak (keycloak-26.6.2).
2. Ubicación del archivo de configuración del realm openiotdata-realm.json dentro del directorio de importación interna:

keycloak-26.6.2/data/import/openiotdata-realm.json

3. Variables de inicio de sesión de administración y ruta de acceso asignadas para el arranque local:
    - KC_BOOTSTRAP_ADMIN_USERNAME=admin
    - KC_BOOTSTRAP_ADMIN_PASSWORD=admin-local-only
    - KC_HTTP_RELATIVE_PATH=/auth

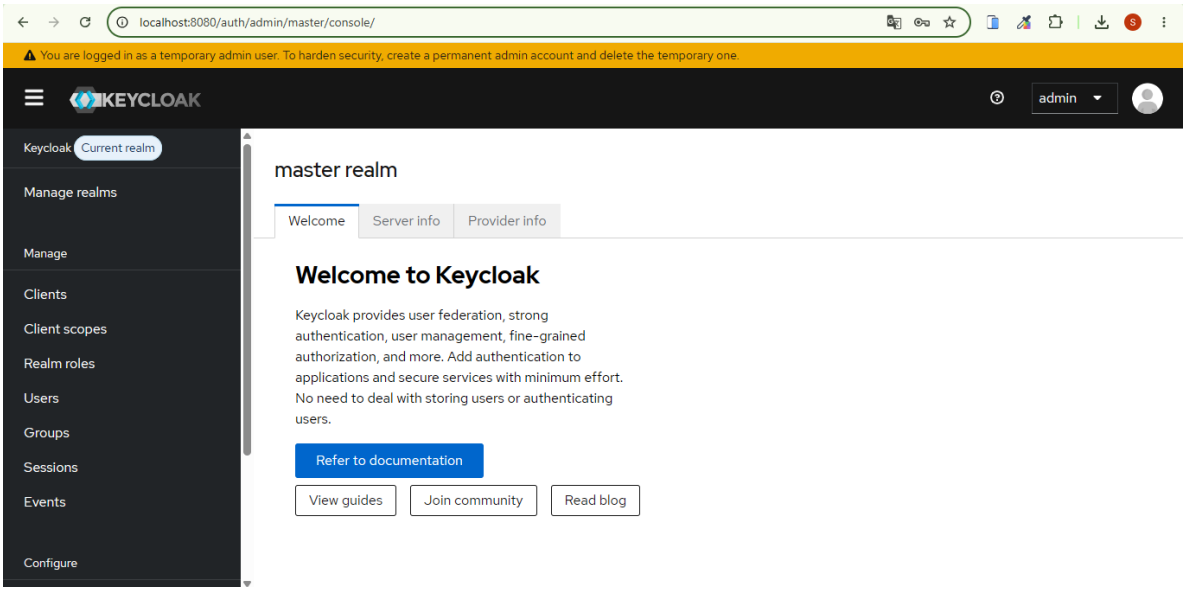
Paso 3: Comando de Ejecución

Ejecución del binario en modo desarrollo cargando automáticamente la configuración del realm:

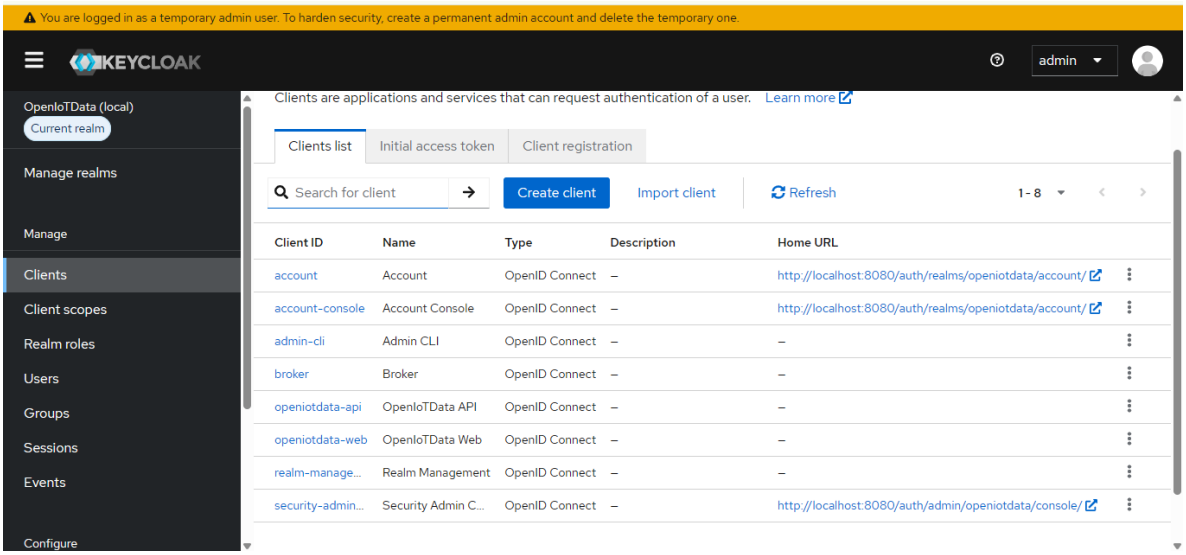
rutaDondeDescomprimioKeyCloak.\bin\kc.bat start-dev --import-realm

5. Estado Actual del Sistema

- **Servicio Keycloak:** Activo en http://localhost:8080/auth.



- **Panel de Administración:** Accesible mediante las credenciales admin / admin-local-only.
- **Realm Cargado:** openiotdata funcional, con sus roles, clientes y usuarios demo habilitados.



- **Consumo de Recursos:** Optimizado para la capacidad total de 4 GB del equipo local.

6. Creación del Auth Provider

6.1. Tipados e Interfaces (AuthUser y AuthContextValue)

- **AuthUser**: Define la estructura de datos normalizada del usuario autenticado (identificador subject, displayName, correo electrónico opcional y el listado de roles asignados).
- **AuthContextValue**: Contiene el contrato del contexto global de autenticación, exponiendo el estado actual (ready, authenticated, user, mode) y los métodos de control (login, register, logout).

6.2. Mapeo de Claims de Keycloak (userFromClaims)

- Procesa los metadatos tokenizados (KeycloakTokenParsed) devueltos por el servidor de identidad.
- Extrae de forma segura el identificador único (sub), el nombre preferido o de pila, y mapea los roles específicos del ámbito del *realm* (realm_access.roles), asegurando que solo se incluyan strings válidos.

6.3. Lógica del Componente AuthProvider

- **Modo de desarrollo (*Bypass* local)**: Permite omitir la autenticación real mediante la variable de entorno NEXT_PUBLIC_DEV_AUTH_BYPASS, inyectando un usuario simulado con roles de prueba (publisher, moderator) para agilizar las pruebas locales.
- **Inicialización de Keycloak (useEffect)**:
    - Configura la instancia de keycloak-js apuntando a las variables de entorno correspondientes (url, realm, clientId).
    - Utiliza el método .init() configurado con la estrategia onLoad: "check-sso" y el flujo **PKCE (S256)** para mayor seguridad en aplicaciones de una sola página (SPA), utilizando un mecanismo de redirección silenciosa (silent-check-sso.html).
- **Persistencia y Refresco de Tokens**:
    - Guarda automáticamente el token de acceso activo en el sessionStorage del navegador bajo la clave openiotdata:access-token.
    - Implementa un mecanismo de actualización periódica (updateToken(45)) mediante un temporizador de 30 segundos y el evento de expiración de Keycloak (onTokenExpired) para mantener la sesión activa sin interrupciones.

6.4. Métodos de Acción (login, register, logout)

- **login / register**: Envuelven las llamadas nativas de la librería de Keycloak (client.login y client.register), redirigiendo al usuario a las vistas correspondientes y retornando a la URL actual al finalizar.
- **logout**: Limpia el almacenamiento de sesión local y ejecuta el cierre de sesión centralizado en el servidor de identidad de Keycloak, redirigiendo al usuario a la raíz del dominio (window.location.origin).

6.5. Hook Personalizado (useAuth)

- Expone un hook auxiliar para consumir el contexto de autenticación de forma segura dentro de los componentes de la aplicación, lanzando un error descriptivo en caso de que sea invocado fuera del ámbito del AuthProvider.

7. Arquitectura del Tema Personalizado en Keycloak (openiotdata)

Para estandarizar la identidad visual del portal e integrar el sistema de autenticación de forma nativa con los tokens de diseño de la organización, se desarrolló un tema personalizado denominado openiotdata dentro del motor de autenticación de Keycloak.

7.1. Estructura de Archivos del Tema

El directorio del tema se configuró dentro de la estructura de almacenamiento de Keycloak bajo la siguiente jerarquía:

Plaintext

```
themes/
└── openiotdata/
    └── login/
        ├── theme.properties
        ├── template.ftl
        ├── login.ftl
        ├── register.ftl
        └── resources/
            └── css/
                └── styles.css
```

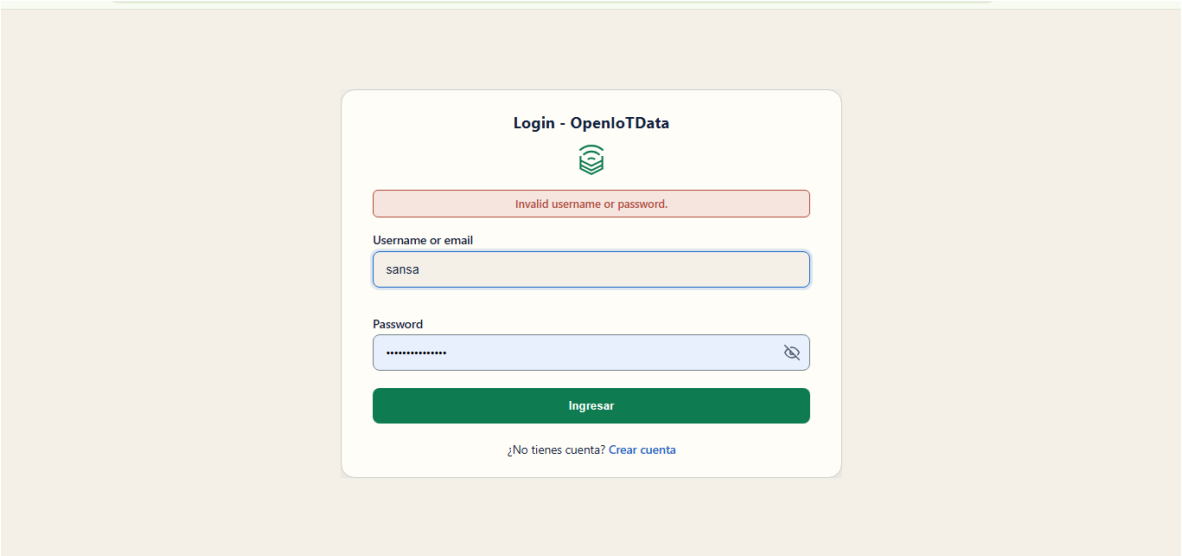
- **theme.properties**: Archivo de configuración base para heredar los recursos de los componentes estándar y enlazar la hoja de estilos global.
- **template.ftl**: Plantilla maestra (esqueleto HTML común) que centraliza la estructura del DOM, la importación de metadatos, favicon, estilos y funciones JavaScript globales.
- **login.ftl**: Vista de inicio de sesión optimizada con diseño centrado de tarjeta y manejo de alertas de error dinámicas.
- **register.ftl**: Vista de registro estructurada mediante un diseño de cuadrícula (*CSS Grid*) de dos columnas para optimizar el espacio vertical.
- **styles.css**: Hoja de estilos centralizada basada en variables de diseño (*design tokens*).

8. Configuración de la Plantilla Maestra (template.ftl)

El archivo template.ftl actúa como el contenedor estructural de todas las vistas del flujo de autenticación. Se diseñó para inyectar dinámicamente los formularios particulares mediante la directiva <#nested "form"> e incorpora un script JavaScript global para gestionar la visibilidad de las contraseñas.

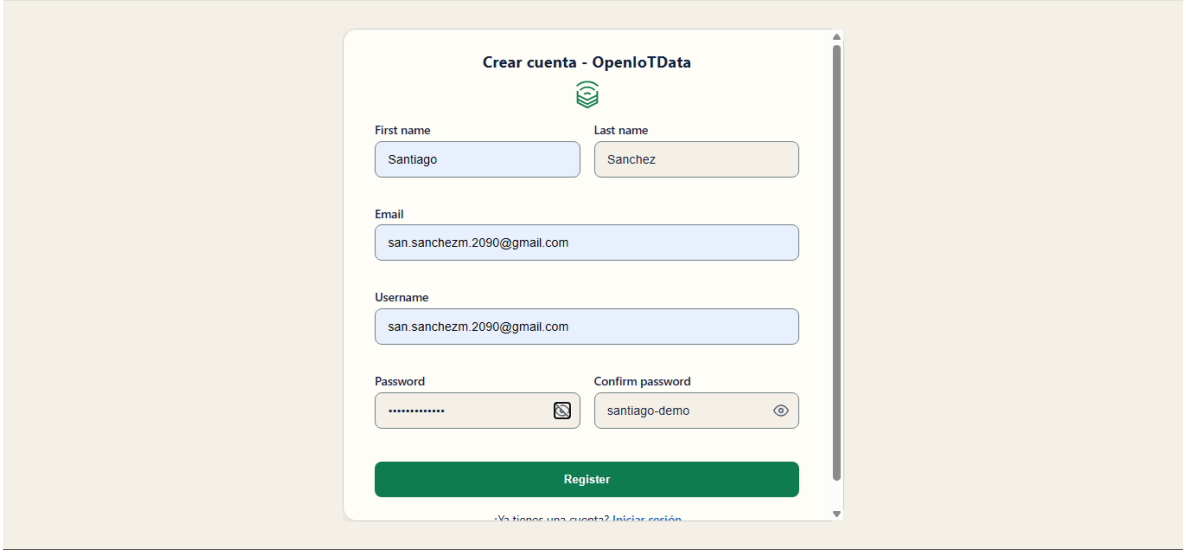
9. Desarrollo de la Vista de Inicio de Sesión (login.ftl)

La vista de acceso implementa un contenedor centrado en forma de tarjeta flotante, suprime los encabezados nativos redundantes de Keycloak e integra un bloque compacto de alertas condicionales para manejar errores de autenticación (por ejemplo, credenciales inválidas o usuarios inexistentes) utilizando la paleta de tonos rojizos de la marca (--vermilion).



10. Desarrollo de la Vista de Registro (register.ftl)

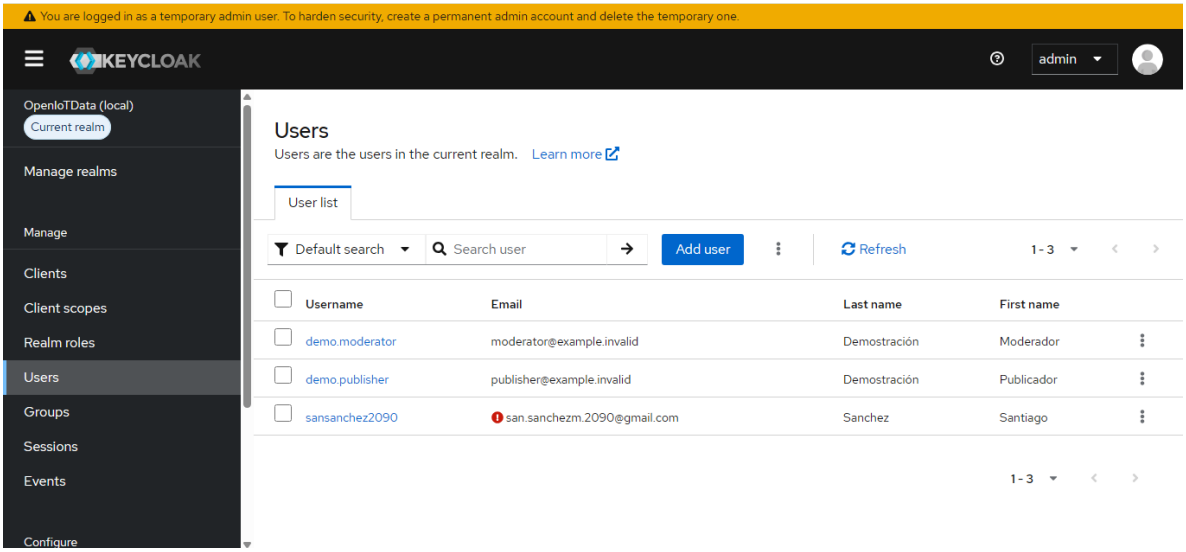
Para resolver de manera óptima la cantidad de campos requeridos en el alta de usuarios (Nombre, Apellido, Correo, Nombre de usuario y Contraseñas), se estructuró un diseño en **cuadrícula de dos columnas (CSS Grid)**, reduciendo la altura vertical del formulario e incorporando de forma nativa los botones de visibilidad de contraseña en ambos campos de clave.



11. Hoja de Estilos y Tokens de Diseño (styles.css)

El archivo CSS consolida los estilos de tipografía, variables cromáticas corporativas, la estructura de la tarjeta central, el comportamiento responsivo del grid y la distribución de los componentes interactivos.

12. Comprobación de la creación de usuario desde el panel de Administrador de KeyCloak



- Inicio de Sesión con el Usuario creado:

